

LKM Rootkit Detection

Can a kernel rootkit stay hidden from an enterprise EDR?

First Year Cybersecurity · University Project · May 2026

Linux Mint 22.3 · Kernel 6.17.0-22-generic · Elastic Security EDR · Diamorphine

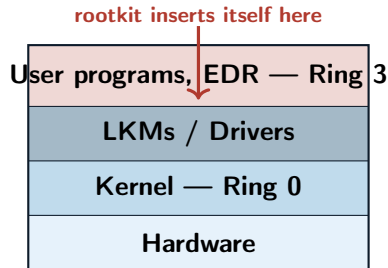
Background — how a rootkit gets into the kernel

The relevant OS structure

The Linux kernel runs in **Ring 0** — the highest privilege level. All user programs, including security tools, run in **Ring 3** and can only access the system by making **system calls**, which the kernel answers.

Linux also supports **Loadable Kernel Modules (LKMs)**: code that can be inserted into the running kernel without a reboot. This is how device drivers work.

A rootkit loads as an LKM, reaches Ring 0, and patches the kernel so that any query about loaded modules returns a filtered list. **The OS is compromised — every tool that asks it gets the same manipulated answer.**



Test environment

- Linux Mint 22.3, kernel 6.17.0-22-generic
- Elastic Security: Elasticsearch + Kibana + Elastic Agent
- Full production-style install — enrollment, policy, detection rules active
- Rootkit: Diamorphine (open source LKM)

EDR verification

EICAR test file dropped on the VM — Elastic flagged it immediately. The EDR works correctly. Missing the rootkit is by design, not a fault.

Findings — what happened after the rootkit loaded

Observed behaviour

Diamorphine loads with one command: `sudo insmod diamorphine.ko`. The hook is active before the command even returns.

`lsmod` — 114 modules. Diamorphine not listed.

Kibana — 0 new alerts.

Kernel reality — 115 modules loaded. Diamorphine is running.

Every tool that asked the OS got the same filtered answer. None of them could tell.

`lsmod / /proc/modules`
114 modules — diamorphine absent

↓ same hook

Elastic Security (Kibana)
0 new alerts

↑ hidden

Kernel reality
115 modules — diamorphine live

Detection method — why the inconsistency exists

What happens when any module loads

When a module loads, the kernel registers it in **two completely separate places**.

First, it adds it to an internal **linked list** — a chain of all loaded modules in memory. This is what `/proc/modules` reads from.

Second, it creates a **directory** for it under `/sys/module/`. This is managed by a completely separate part of the kernel called `sysfs`. The two never communicate with each other.

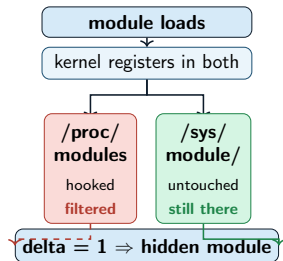
Diamorphine was added to both when it loaded. The kernel always does this and Diamorphine does not stop it.

What Diamorphine does after loading

After loading, Diamorphine hooks `getdents64` — the system call the kernel uses to list directory entries. Every time anything reads `/proc/modules`, Diamorphine intercepts that call and strips its own entry from the response before it is returned.

It never touches `/sys/module/`. Its directory remains there.

The module is not gone — it is being filtered on every read. The two paths disagree by one entry. That is the detection signal.



detector.py — what it does and what it found

How the detector works

The detector is a Python script, with no external libraries needed.

It does three things:

1. Reads the module list from `/proc/modules`
2. Reads the directory list from `/sys/module/`
3. Compares the two. Anything in `/sys/module/` that is not in `/proc/modules` is being hidden.

In a clean system the two lists always match. When Diamorphine is running they differ by one. That one missing entry is Diamorphine.

```
$ sudo python3 detector.py
kernel 6.17.0-22-generic
proc=114 sysfs=115 delta=1

hidden modules detected

diamorphine
initstate : live
sysfs    : /sys/module/diamorphine/
procfs   : absent -- filtered

verdict:  SYSTEM COMPROMISED
```

proc=114 sysfs=115 **delta=1**
One module in sysfs is not in proc.
That module is Diamorphine.

What the demo shows

Step 1 — Baseline. Detector runs on a clean machine and reports clean. `lsmod` and Kibana both agree.

Step 2 — Prove the EDR works. An EICAR test file is dropped on the VM. Elastic catches it immediately in Kibana.

Step 3 — Load rootkit. `sudo insmod diamorphine.ko`. `lsmod` still shows 114 modules. Kibana shows zero new alerts.

Step 4 — Run detector. `proc=114 / sysfs=115 / delta=1`. Diamorphine named and located.

Demo

Limitations & Future Improvements

Current limitations

This is a **proof of concept** — it exploits a specific gap that Diamorphine left open by not patching `/sys/module/`.

A more complete rootkit that also patches `sysfs` would not be detected by this method. At that point a different independent reference would be needed.

The detector currently targets **LKM-based rootkits only**. Other techniques — memory-only implants, eBPF-based rootkits, bootkits — are outside its scope.

Future improvements

The project is still under development. Planned directions:

- Cross-reference additional kernel data structures beyond `/proc` and `/sys`
- Extend detection to other rootkit families and evasion techniques
- Harden against rootkits that patch both paths

Summary

A kernel rootkit (Diamorphine) was loaded on a VM running Elastic Security EDR. The rootkit hooked `getdents64` to hide itself from `/proc/modules`. Elastic reported no alerts. A cross-view detector comparing `/proc/modules` against `/sys/module/` identified the hidden module correctly. The detection gap exists because `sysfs` is managed by a separate kernel subsystem that the rootkit did not patch.